DP3. Exact integer time, wrap-exact whole-number phases and bit-identical CPU and GPU fixed-point twins, with a network mode that sends laws of motion instead of states

Defensive publication · draft 2 · 23 October 2026 · Author: Kyle Kimberlin (AI assistants helped write it) · Licence: CC BY 4.0

Keywords: deterministic simulation; bit-exact; reproducible; cross-platform; cross-vendor GPU; integer time; simulation clock; epoch; fixed time step; sub-step fraction; predicted display time; head-mounted display; virtual reality; binary angle; phase accumulator; wrapping multiplication; whole-number frequency; periodic motion; procedural animation; ocean waves; wind; tree sway; fixed-point arithmetic; sine lookup table; CPU and GPU twins; parity test; compute shader; SPIR-V; networked game; multiplayer; state synchronisation; dead reckoning; lockstep; rollback; motion law; random seed; replay; persistence.

Classification (CPC): A63F 13/57 (simulating properties, behaviour or motion of objects in the game world); A63F 13/35 (details of game servers); H04L 67/131 (protocols for games, networked simulations or virtual reality); G06T 13/60 (3D animation of natural phenomena, e.g. rain, snow, water or plants); G06F 1/03 (digital function generators working by table look-up); G06F 7/548 (trigonometric functions; co-ordinate transformations).

Related disclosures in this series: DP2, caches certified by laws of time, and drivers and frames · DP4, sound made from the matter (sound laws on the same integer phase) · DP5, the physel record and drawing without meshes.

1. Field

Simulation that several machines must compute alike, in games, virtual reality, film, distributed simulation, digital twins and robotics: representing time and periodic motion so that every CPU and GPU produces the same bits, and sending and storing motion as laws of time instead of samples.

2. Problem and known methods

Most of what moves in a large shared world is predictable: the sun, wind, waves, swaying plants, doors, lifts, ferries. If every machine computed that motion from a few shared numbers and got exactly the same answer, nothing would need streaming or drift, and a late joiner could start at once. Four things stand in the way.

1. **Time built by adding frame times drifts.** `t = t + dt` rounds at every frame, by amounts that depend on the frame rate, so machines at 72 Hz and 90 Hz drift apart. A step of 1/480 s has no exact binary form.
2. **Floating-point time loses resolution far from its epoch.** Seconds in 32-bit floating point step by 2 s a year after the epoch. Double precision is fine-grained (about 0.24 µs for seconds since 1970), but see the next point.
3. **Functions of time are not the same bits everywhere.** IEEE 754 fixes the basic operations, but library sine, exponential and logarithm need not be correctly rounded and differ between platforms (Monniaux 2008). GPU interfaces allow reassociation, fused multiply-add, flushing of tiny values, divisions within 2.5 units in the last place and arc tangents within 4,096 (the Vulkan specification). The same shader gives different bits on different vendors' GPUs.
4. **Sending motion costs bandwidth or accuracy.** State synchronisation sends each moving object many times a second. Dead reckoning (IEEE 1278) extrapolates between updates, with

error thresholds and visible corrections.

Known method	What it lacks for this purpose
Lockstep: inputs sent, every machine recomputes (Bettner and Terrano 2001; Fiedler 2014)	Needs bit-exact simulation everywhere; usually CPU only; a late joiner replays or downloads
Rollback (GGPO; Stallone 2018); optimistic simulation (Jefferson 1985)	The same need; no rule for time itself or for GPU work
Cross-platform CPU physics with its own trigonometry (Catto 2024)	GPU work is outside it
Reproducible floating-point sums (Collange et al. 2015)	Sums only: not functions, not time
Integer time units: NTP timestamps (RFC 5905), IEEE 1588, rational media time bases, the "flick" (1/705,600,000 s)	No phase form a 32-bit GPU can use exactly; no rule for a display time between simulation steps
Phase accumulators in direct digital synthesis (Tierney, Rader and Gold 1971); binary angles and sine tables in early games (Doom, 1993)	The phase is a running sum, so its value at an arbitrary time needs the whole history; no GPU twin
Looping procedural motion as a closed function of time: Tessendorf (2002) rounds wave frequencies to multiples of a base frequency; Stam (1997) shapes noise in the frequency domain for plants	Floating point, so not bit-identical across devices; the phase is a float product that loses precision far from zero

What is missing is one representation of time and phase that is never summed and works before and after its epoch. It should carry a sub-step fraction for display timing, give exact periodic phases with 32-bit integer operations, and come with functions that give the same bits on CPUs and GPUs of every vendor. Also missing is a network and storage scheme, built on it, that sends and keeps laws instead of samples.

3. Summary

Time is one signed 64-bit count of 1/480 s steps from a fixed UTC instant, plus an 11-bit fraction used to draw at the display's predicted time. It is never built by adding frame times, and each world maps it to its own time scale. Every periodic motion has a whole number n of cycles in a period of 2^{21} steps, so its phase in 2^{-32} turns is one wrapping 32-bit product, n × t32, where t32 is the time's low 32 bits in 1/2,048 steps: exact on every CPU and GPU. Sine, cosine, 2^{-x}, log2 and square root use integers and tables made by one integer-only generator, with a CPU twin and a GPU twin (32-bit words only) that are tested for equal bits. A network mode, LAW, sends a law's kind, parameters, start and seed once; every machine evaluates the same bits at its own display time; non-player characters travel as intents, breaks as commands, and full body poses never. Results are sorted into three layers (gameplay on the CPU in double precision, shared integer forms, cosmetic matter), and a world stores laws, seeds, recipes and causes instead of samples.

4. Detailed description

4.1 What runs where

The CPU alone runs the clock, world clocks, exact non-power-of-two periods, the fine phase, the table generator and every law maker. The CPU and the GPU both run, as twins, the phases, the fixed-point functions, law evaluation and poses. The network, the gameplay layer and storage run on the CPU.

4.2 The time value (Figure 1)

Field	Type	Bytes	Meaning
step	Signed 64-bit integer, two's complement	8	Whole steps of 1/480 s since the epoch E0 = 2026-01-01 00:00:00 UTC (Unix time 1,767,225,600 s); negative before E0
fraction	Unsigned 16-bit integer	2	The part of a step, in 1/2,048 step = 1/983,040 s, about 1.017

Field	Type	Bytes	Meaning
	holding 0-2,047 (11 bits)	µs	

Rules:

1. The count is in SI seconds and is never stepped for leap seconds.
2. The range is $\pm 2^{63}$ steps, about $\pm 6 \times 10^8$ years. Times before the epoch are valid everywhere (a world's history, generated pasts).
3. A tick is 1/2,048 step: ticks = step × 2,048 + fraction, in 128-bit integers, exact for every time. A second has 983,040 ticks. Times are ordered by step, then fraction, which is the order of their ticks.
4. Every division rounds down (floor), so negative times behave like positive ones. The 120 Hz tick that network hosts share is ⌊step ÷ 4⌋: step −1 is in tick −1.
5. Times are never summed. The distance between two times is an exact difference of ticks. Frame k of a fixed-rate run is a product: start + ⌊k × 983,040 ÷ fps⌋ ticks.
6. Floating point appears only at a boundary: seconds for people (or for readers not yet converted) are $(\text{ticks}_1 - \text{ticks}_0) \div 983{,}040$, turned into double precision once; a device's display time becomes the nearest tick once. A source-scan test refuses floating-point types and functions everywhere else in the time and law code.
7. Unix time converts with integers only: ticks = (seconds − 1,767,225,600) × 983,040 + ⌊nanoseconds × 3,072 ÷ 3,125,000⌋ ($983{,}040 \div 10^9$, both divided by 320). Back: seconds = ⌊ticks ÷ 983,040⌋ + 1,767,225,600 and nanoseconds = ⌈r × 3,125,000 ÷ 3,072⌉ with r = ticks mod 983,040, so a round trip returns the same time.

Why these values: 1/480 s divides the 1/120 s network tick and every rung of a power-of-two ladder of solver steps up to 1/30 s. It holds exactly 100 samples of 48 kHz audio. The fraction exists for drawing, because a headset shows a frame at a time that does not fall on a step. Rounding to the nearest step would misplace a moving object by up to 1.04 ms: at 10 m/s and 1 m that is 0.6° of view, about 9 pixels on a display of about 15 pixels a degree. Eleven bits make the phase word exactly 32 bits (§4.4).

The frame clock. Each frame has one time, *now*, and the last frame's time, *prev*. *now* comes from one of three sources, each read through one anchor (a time paired with a reading of the source), never by adding:

Source	*now*
Fixed frames (tests, offline runs)	start + ⌊k × 983,040 ÷ fps⌋ ticks for frame k (for a frame period that is no whole rate: k × the period in ticks)
Real time (a window)	The host's UTC read once at start, in ticks; then the anchor + the monotonic clock's elapsed nanoseconds in ticks
A headset	The anchor + (predicted display time − the display time at the anchor), turned into ticks once

A source that takes over is anchored at the last frame's time plus its own step (one frame period, one display period, or the time since the last frame). *now* never goes back: *now* = max(*now*, the last frame's *now*).

The world clock. Each world states a rate p/q of epoch time and an anchor: world time = (the world time at the anchor) + ⌊(t − anchor) × p ÷ q⌋ ticks, in 128-bit integers, rounded down, also before the anchor. p = 0 holds a world still; p/q = 24 gives a one-hour day. **Each law names its clock:** world time (the sun, growth, weather) or physical time (cooling, sway, drying), so a world with a fast day keeps physics at its real rate.

4.3 Drawing at the display time, once for both eyes

1. Once a frame the engine asks the headset runtime for the frame's predicted display time (as OpenXR provides).
2. It maps that to *now* (§4.2): a step and a fraction.
3. *now*, and the world clock's reading at *now*, go to every pass as integers: per clock a 16-byte vector of 32-bit words (step low word, step high word, fraction, 0). Shaders never receive floating-point

time.

4. Every law, phase and pose of the frame is evaluated at that one time, for both eyes: they see one pose, at the runtime's predicted display time, stored to about 1 μs.

4.4 Whole-number phases (Figure 2)

Rule	Value
Period	T = 2^{21} steps = 2,097,152 ÷ 480 s, about 4,369.07 s or 72.8 min
Ticks in a period	$2^{21} \times 2^{11} = 2^{32}$
Frequency	n whole cycles per T, n an unsigned 32-bit integer; f = n × 480 ÷ 2^{21} Hz; resolution 1/T, about 0.229 mHz
Choosing n	n = round(f × 2^{21} ÷ 480). The angular frequency is off by at most π/T, about 0.00072 rad/s: about 0.029% for a deep-water wave 10 m long (ω about 2.48 rad/s)
Time word	t32 = (step × 2^{11} + fraction) mod 2^{32}: the step's low 21 bits shifted left by 11, then the fraction in the low 11 bits
Phase	phase32 = n × t32 + φ, a wrapping 32-bit multiply and add, in 2^{-32} turns; φ is the phase at the epoch

Why it is exact. The phase in turns is n × ticks ÷ 2^{32}. Its fractional part in 2^{-32} turns is (n × ticks) mod 2^{32}. Because (a × b) mod 2^{32} = (a × (b mod 2^{32})) mod 2^{32}, only ticks mod 2^{32}, which is t32, matters, and a 32-bit unsigned multiply keeps exactly the low 32 bits of the product. Every CPU and every GPU computes this the same way, with no rounding. Two's complement makes it hold before the epoch too. A period later t32 is the same, and so is every phase: whole-number-phase motion repeats exactly every 72.8 minutes.

Pseudo-code: a sum of waves.

```
value(waves, c, t):                    // t = (step, fraction) on the law's clock
    t32 = (low32(step) << 11) | fraction
    sum = 0                            // signed 64-bit
    for (a, n, phi) in waves, in stored order:
        p = n * t32 + phi              // unsigned 32-bit, wrapping
        sum = sum + a * sin_q30(p)     // 32 x 32 -> 64-bit product
    return clamp_i32(c + (sum >> 30)) // arithmetic shift
```

Periods that are not powers of two (a day, a tide, a season, any whole number P of steps): phase = ⌊((step mod P) × 2,048 + fraction) × 2^{32} ÷ (P × 2,048)⌋, in 2^{-32} turns, rounded down. It needs a division, so the CPU works it out once a frame and hands shaders an integer phase. Below 2^{37} steps (about 9 years) it fits in 64 bits: with d = P × 2^{11} (below 2^{48}), x = (step mod P) × 2^{11} + fraction (below d), a = x × 2^{16}, h = ⌊a ÷ d⌋ and r = a mod d, the phase is h × 2^{16} + ⌊r × 2^{16} ÷ d⌋. Longer periods use 128-bit integers. The sun, moon and planets come from an orbit law, not from a whole-number period.

The fine phase for sound (CPU only). At 48 kHz a step holds 100 samples, and the 11-bit fraction is too coarse for tones above about 1 kHz. The fine phase takes a 32-bit fraction of a step, q (in 2^{-32} step): phase = ⌊n × ((step mod 2^{21}) × 2^{32} + q) ÷ 2^{21}⌋ mod 2^{32}, in 128-bit integers. With q = fraction × 2^{21} it equals n × t32 exactly. Sample k of a step has q = ⌊k × 2^{32} ÷ 100⌋.

4.5 Fixed-point functions and their tables (Figure 3)

Q1.30 is a signed 32-bit integer v meaning v ÷ 2^{30} (one is 2^{30}). Q32.32 is an unsigned 64-bit integer meaning v ÷ 2^{32}.

Function	Takes	Gives	Method	Bound
sin(p)	A phase p: unsigned 32-bit, 2^{-32} turns	Q1.30	Table SIN of 4,097 entries, sin(2πi ÷ 4,096) for i = 0 to 4,096 (the last repeats the first, so no wrap). i = p ÷ 2^{20} (the top 12 bits), f = p mod 2^{20}; SIN[i] + ⌊(SIN[i+1] − SIN[i]) × f ÷ 2^{20}⌋	3.2×10^{-7}
cos(p)	As sin	Q1.30	sin(p + 2^{30}), wrapping	3.2×10^{-7}

Function	Takes	Gives	Method	Bound
2^{-x}	$x \geq 0$ in Q32.32	(m, k): $2^{-x} = m \times 2^{-(30+k)}$, m in Q1.30 within $[2^{29}, 2^{30}]$, k the whole part of x	If $k \geq 64$ the result is (0, 64): zero by a test, never by a shift of 64 or more. Else table EXP2 of 1,025 entries $2^{-(j \div 1{,}024)}$: j = the top 10 bits of x's fraction, interpolated on the next 20	2×10^{-7} relative
log2(x)	Unsigned 64-bit x	Q8.24 (2^{-24} units); the least 32-bit value for x = 0	e = the place of the leading one bit. Of the 30 bits after it, the top 10 pick j in table LOG2 of 1,025 entries $\log_2(1 + j \div 1{,}024)$ and the next 20 interpolate; result = $e \times 2^{24}$ + (the interpolated value $\div 2^6$)	2^{-20} (9.54×10^{-7})
isqrt(x)	Unsigned 64-bit x	$\lfloor \sqrt{x} \rfloor$, 32 bits	32 rounds from the top bit: set the bit in r; keep it if $r^2 \leq x$	Exact
64-bit helpers	A 64-bit word and any shift count	Shifts; saturation to 32 bits	A shift of 64 or more gives 0, or the sign for an arithmetic shift, chosen by a comparison	Exact

The sine's bound is that of linear interpolation, $h^2/8$ with $h = 2\pi \div 4{,}096$ (about 2.94×10^{-7}), plus rounding.

Interpolating without 64-bit products on the GPU. Neighbouring entries differ by less than 2^{21} (checked when the tables are made). With d = SIN[i+1] − SIN[i] and f = fh × 2^{10} + fl (fh and fl below 2^{10}), the GPU computes $\lfloor (d \times fh + \lfloor d \times fl \div 2^{10} \rfloor) \div 2^{10} \rfloor$ with arithmetic shifts. Each product stays below 2^{31}, and the result equals $\lfloor d \times f \div 2^{20} \rfloor$ for every input, because $\lfloor (x + \lfloor y \rfloor) \div m \rfloor = \lfloor (x + y) \div m \rfloor$ for whole numbers x and m.

The table generator. One program makes all three tables with integer arithmetic only, at 2^{-62} in 128-bit integers:

1. π at 2^{-124} by Machin's formula, 16 arctan(1/5) − 4 arctan(1/239), each arc tangent by its series, every term rounded down.
2. ln 2 at 2^{-124} as 2 artanh(1/3), the sum of $1 \div ((2k + 1) \times 3^{(2k+1)})$, doubled.
3. Sine and cosine at the 1,025 points of a quarter turn by their series; the rest of the turn by symmetry, so the entries at quarter turns are exactly 0, 1, 0, −1 and 0.
4. $2^{-(j \div 1{,}024)}$ as e^{-y} with $y = j \times \ln 2 \div 1{,}024$, by its series; $\log_2(1 + j \div 1{,}024)$ as $\ln(1 + z) \div \ln 2$, with $\ln(1 + z) = 2 \operatorname{artanh}(z \div (2 + z))$.
5. Each entry is rounded to the nearest Q1.30 value, after a check that the exact value lies more than 2^{10} units of 2^{-62} from a rounding midpoint (its own error is below about 10^2 units), so the rounding is certain.
6. The tables hold 6,147 entries, 24,588 bytes. A test fails if the stored tables differ from what the generator writes; another checks π and ln 2 to 18 decimal places.

The GPU twin reads the same bytes from a storage buffer that the CPU fills from its own arrays, never from constant arrays compiled into the shader, which a shader compiler may copy into every thread's memory at each use.

4.6 The rules that make the twins agree

Rule	Why
Integers only. On the GPU, 32-bit integer types only: a 64-bit value is a (low, high) pair of 32-bit words with explicit carries and borrows; a full 32 × 32 → 64-bit product comes from four 16-bit partial products, or from the core SPIR-V extended multiplies (OpUMulExtended, OpSMulExtended)	64-bit shader integers are optional on many devices
No division on the GPU; powers of two are shifts; other exact periods come from the CPU	Division can differ between vendors
No shift by the word's width or more: such shifts are chosen by a comparison	SPIR-V leaves them undefined, and some CPU languages mask the count

Rule	Why
No recursion: a composite law is a flat list whose factors are never composites	Shaders cannot recurse
Wrapping arithmetic only where a wrap is meant; checked or wider arithmetic elsewhere	Debug and release builds give the same bits
Products summed in 64 bits, in stored order	The order fixes the bits
No floating point, no fused multiply-add and no hash containers (whose order can vary) in the time and law code; a source-scan test enforces it, with floating point allowed in one boundary module only	The same bits on every machine
Parity means equal bits; a difference is a bug found at its cause, never a tolerance	—

4.7 Laws as words

A law is a closed formula of time on one clock, kept as 32-bit words that the CPU and the GPU read unchanged. Its header is 8 words (32 bytes):

Word	Bits	Holds
0	0-2	The kind
0	3	The clock: 0 physical, 1 world
0	4-9	The exponent e (signed): a value v means v × 2^e
0	10-20	The fraction of the start time t0
0	21-31	The fraction of the end time
1-2	All	The step of t0, low word first
3-4	All	The step of the end
5-6	All	The envelope: lowest and highest value (signed 32-bit)
7	All	The largest rate: the most the value changes in one step (unsigned 32-bit)

Outside [t0, end] a law reads its clock as t0 or the end, so it holds its first and last values. The maker works out the envelope and the rate from the evaluator's own integer operations, rounding and table error included, so they hold for every input (they may be loose). It refuses a law whose values could leave 32 bits inside its window. For a sum of waves the envelope is $c \pm \Sigma|a|$ and the rate is $1 + \lfloor \Sigma |a| \times W(n) \div 2^{30} \rfloor$, with $W(n) = \min(\lfloor 1{,}609 \times d \div 1{,}024 \rfloor + 4, 2^{31})$ and $d = \min(2{,}048 \times n, 2^{31})$, the most a step moves the phase: the sine's steepest slope, $\pi/2$ a unit in Q1.30, is below 1,609/1,024, and the table adds at most 4. Every kind's bounds are given in DP2, "Envelopes and largest rates", where caches use them to know how long a fact stays true.

Kind	Parameters (32-bit words)	Value
Constant	c	c
Polynomial, to cubic	One word: s, f1, f2, f3 at bits 0, 8, 16, 24; then c0-c3	τ = (ticks since t0) >> s; c0 + ((c1 + ((c2 + (c3 τ >> f3)) τ >> f2)) τ >> f1), each stage a 64-bit product rounded down and held to 32 bits
Sum of waves	Count; c; per wave a, n, φ	$c + \lfloor \Sigma\, a \times \sin(n \times t32 + \varphi) \div 2^{30} \rfloor$ (§4.4)
Damped modes	One word: count (bits 0-15), s (16-21), r (24-29); c; per mode a, n, φ, λ	The sum of waves with each amplitude decayed from t0 by (λ × τ) >> r halvings through 2^{-x}, rounded toward zero so a decay ends at exactly 0. A half-life of h ticks gives λ = 2^(32 + r + s) ÷ h
Relaxation in bands	Count; per band its start (ticks after t0, 2 words), from, toward, λ, and s and r	toward + (from − toward) decayed, in the last band started (cooling, drying)
Periodic table	One word: k (bits 0-5) and m (bits 8-12), with m ≤ 16, m ≤ k + 11 (no sample shorter than a tick) and k ≤ 53; then 2^m samples	The samples spread over 2^k steps from t0, repeating: the top m bits of the position's k + 11 bits pick a sample, and the next 20 interpolate toward the next sample
Composite	Count; per term a law's word index, a second law's index or none, and a signed shift	The sum of products, shifted and held to 32 bits; flat
Modal instance	No header. Word 0: the kind (bits 0-2), t0's fraction (3-13), a frequency factor k in 2^{-16},	The archetype's law, read with its clock, end, exponent, s and r, at the instance's own t0 and c.

Kind	Parameters (32-bit words)	Value
	1 to $2^{18} - 1$ (14-31); words 1-2: to's step; word 3: where its archetype, a damped law in the same table, starts; word 4: c; then one word per mode: a share of the archetype's amplitude A in 2^{-15} (bits 0-15, signed) and a phase in 2^{-16} turn (bits 16-31)	Per mode: a = (A × share) >> 15, n' = (n × k) >> 16, λ' = (λ × k) >> 16, and the phase n' × t32 + (the mode word with its low 16 bits cleared), which replaces the archetype's φ. A smaller copy sways faster and decays in step; 9 words for four modes. Its bounds are the expanded law's, worked out on the CPU

A four-mode damped law is 26 words (104 bytes). Further kinds may be added: path laws (splines, speed profiles, timetables) and orbit laws (the sun to the accuracy of a published solar-position algorithm).

Poses from laws. A moving frame's local pose is made in integers by one of three recipes, each 12 words: fixed (the rest pose), hinge (a turn by an angle law about an axis through a pivot) and loop (a body going round a circle at n whole turns a period, with an optional bob law). DP2, "Local poses in integers, then f32 in one order", gives their words and poses. A rotation is a Q1.30 quaternion from the sine and cosine of the half angle, made unit by one integer Newton step (g = (3 × 2^{60} − the squared length) ÷ 2^{31} in Q1.30, then each part × g ÷ 2^{30}); offsets are in 2^{-24} m. Where a pose must become 32-bit floating point for drawing:

1. each part is rounded to 24 significant bits in integers, so it converts exactly;
2. the parts compose in one fixed order, and every float multiply, add and subtract is marked so that no compiler fuses or reorders it (SPIR-V's NoContraction decoration on the GPU; no fused multiply-add on the CPU), with no square root and no normalising;
3. any part below 2^{-126} is written as +0 on both twins, since a GPU may flush it.

So the CPU and the GPU produce the same floats.

Events that decide results (a law ends, a threshold is crossed) wait in one queue per area, ordered by step, kind and a stable id, the same on every machine. How they are found belongs to DP2.

4.8 The LAW network mode (Figure 4)

The shared world's protocol sends each object in one of five modes:

Mode	Sent as
SILENT	Nothing, after one reliable rest state
CAUSE	Only its cause: every machine computes the object in the gameplay layer; a rest pose and a hash confirm it
STATE	Delta-coded state, latest only
HELD	Inside the holder's pose
LAW	A law's kind, parameters, start and seed, once

Steps:

1. The authority (a host) decides that an object moves by a law: a door opens, a lift is called, a ferry leaves, the weather changes.
2. It sends one LAW message on the reliable, in-order stream, stamped with the 120 Hz tick of its cause.
3. Each receiver rebuilds the law's words with the same integer maker. The maker also works out the envelope and the largest rate, so these are never sent.
4. Each receiver evaluates the law at its own predicted display times (§4.3) and, where gameplay reads it, at tick boundaries. Every machine computes the same bits from the same words at the same time, so there is no dead-reckoning threshold and no correction.
5. A late joiner receives the area's law table in its snapshot and evaluates every law at the join step.
6. A change of law (the door is stopped, the ferry is delayed) is a new cause: a new LAW message with a new start.
7. When the object is blocked or touched it leaves LAW for CAUSE or STATE.

One possible encoding (an example):

Field	Type	Bytes	Notes
Message kind	Unsigned 8-bit	1	LAW
Object handle	Unsigned 32-bit	4	An area-local index and generation; lasting names are stable part ids
Tick	Unsigned 32-bit	4	The cause's 120 Hz tick, relative to the last acknowledged tick
Law head	Unsigned 16-bit	2	Kind (3 bits), clock (1 bit), exponent (6 bits, signed), parameter word count (6 bits)
Start	Signed 32-bit	4	t0 in ticks from the cause's tick (±36 minutes)
Length	Unsigned 32-bit	4	Ticks from t0 to the end; all ones for "until changed"
Seed	Unsigned 64-bit	0 or 8	Present when the parameters are generated from a seed
Parameters	32-bit words	4 each	The kind's parameter words

A door that swings open in 1.5 s as one cubic segment (a shift word and four coefficients) takes 1 + 4 + 4 + 2 + 4 + 4 + 20 = 39 bytes, sent once.

Seeds. A law may be generated rather than listed. A swell is a spectrum and a seed. The maker runs on the CPU in integers (where exact integer division is also allowed), so every machine makes the same waves:

1. The parameters hold the spectrum's inputs in fixed point (for example the wind speed U), a band of whole-number frequencies from n_lo to n_hi, a wave count N and a 64-bit seed.
2. Wave i takes two 64-bit words, r0 and r1, from a counter-based generator keyed by (seed, i) (Salmon et al. 2011; Steele et al. 2014). It needs no other state, so waves can be made alone and in any order.
3. Its frequency: with $w = \lfloor (n_hi - n_lo) \div N \rfloor$, $n_i = n_lo + i \times w + \lfloor \lfloor r0 \div 2^{32} \rfloor \times w \div 2^{32} \rfloor$, a whole number of cycles a period in the i-th part of the band.
4. Its amplitude: $a_i = \sqrt{(2\, S(\omega_i)\, \Delta\omega)}$, with $\omega_i = 2\pi\, n_i \div T$ and $\Delta\omega = 2\pi\, w \div T$, worked out through logarithms: log2 S from §4.5's log2 and integer products, S from 2^{-x} and a shift, the root by the integer square root. The Pierson-Moskowitz spectrum, $S(\omega) = \alpha\, g^2\, \omega^{-5} \exp(-\beta\, (g \div (U\, \omega))^4)$, gives $\log_2 S = \log_2(\alpha\, g^2) - 5 \log_2 \omega - \beta\, (g \div (U\, \omega))^4 \log_2 e$.
5. Its phase: $\varphi_i = \lfloor r1 \div 2^{32} \rfloor$, in 2^{-32} turn.
6. The waves become a sum-of-waves law (count N, c = 0, then each a_i, n_i, φ_i), with an exponent that keeps Σ|a| within 32 bits.

Any spectrum works if it is evaluated with these integer functions. For a surface, each wave also draws a direction; its phase at a point adds the dot product of its wave vector (in 2^{-32} turn a metre, fixed point) and the position, with a wrapping add. The sun, sky, wind, swell and the sway of plants then need no messages: they come from the clock and from seeds that are part of the world.

4.9 Intents, commands and bodies

- **Non-player characters travel as intents** in CAUSE mode ("walk route R from a to b from tick N"). Every machine runs the same CPU gameplay layer: the route, the gait phase as a law between decisions, and a new decision only on contact or a new intent.
- **Bodies are never sent as full poses.** Every machine computes the same body from the inputs it is given: a player's tracked head and hands (masked before sending), or a character's intent. Head and hand motion alone can identify a person (Nair et al. 2023), so less is sent and less is kept.
- **Breaks are sent as commands** (Figure 5): which bonds failed (or where a blast was, and its energy), the tick, and an 8-byte hash of the outcome. The pieces are made from the command by **unfolding**: content stored compactly (a wall as a lattice of bricks, a pane as a lattice of dice) becomes real pieces only in the cells the command names. A generator runs on the CPU in integers, with counter-based random numbers keyed by each part's path (such as "course 17, slot 5"), never by the order parts are made in. Elements below a chip size become grains, and each

piece is named by its parent, the command's number and its own index. So every machine makes the same pieces. Pieces that matter to gameplay move in the gameplay layer and are confirmed by a hash once they rest; small debris moves in the cosmetic layer, where its motion may differ between machines. Which cells fail is outside this disclosure. A far or hidden player gets only the lasting result as an edit, never the debris.

4.10 The three determinism layers (Figure 5)

Layer	Numbers	Runs on	Promise
Gameplay	Double precision within a tick; signed 64-bit ledgers (mass in micrograms, heat as enthalpy in 2^{-20} J); integer time	The CPU of the host, each client and any referee	Bit for bit on every supported operating system and CPU family (x86-64 and 64-bit ARM), in debug and release builds, at any thread count, from the same source and pinned compiler
Shared integer forms	32- and 64-bit integers and fixed point, summed in a fixed order	CPU and GPU	Bit for bit on every CPU and every GPU vendor
Cosmetic	Single precision	GPU	The same on one build, GPU model and driver; never read by gameplay; never decides a result

A presentation step reads all three and writes none.

Gameplay rule	What
One scalar type	Double precision exposing only what IEEE 754 defines exactly (the four basic operations, square root, absolute value, floor, total order), plus an in-house maths library built from them
Bans	Library maths of unspecified precision, fused multiply-add, CPU-specific code, host clocks, unseeded random numbers, hash containers
State	No pointer-sized integers; encoded little-endian field by field (floats as bit patterns): the same bytes for rollback, snapshots, recordings, the hash and the wire
Order	Integer time (§4.2); a total key for every sort; work split by a fixed grain, never by the number of cores
Builds	Overflow checks kept in release builds; no hidden state; the floating-point mode checked each tick in debug builds; tiny values cleared by thresholds, never flushed
Positions	An integer region id and a double offset in power-of-two regions (4,096 m), changing region only at a tick boundary, by an exact power-of-two shift

Fixed point is kept for anything conserved or shared with the GPU. For gameplay, a 48.16 fixed-point unit (15.3 μm) is too coarse: one step of fall under gravity is only about 42.6 μm, under three units. Double precision's basic operations are exact on both CPU families.

Shared integer forms. Heat as energy: a flux is worked out once per contact or face, +q on one side and −q on the other, in whole numbers, so any order of addition gives the same total. Water ledgers in micrograms. Summary sums over a tree of regions, objects, parts and chunks. Law phases and values, and poses that must agree (§4.4-§4.7). On the GPU: integer atomics only (their final sums are exact in any order); any order that matters comes from prefix sums or sorting, never from the order in which atomics hand out slots; kernels correct at any wave size; and a CPU twin with a parity test for each kernel.

Detecting a split. Debug builds hash the state every tick, part by part; release hosts keep a hash chain every 60 ticks; each predicted object in a snapshot carries a 4-byte hash of its exact state.

4.11 Storing laws, seeds, recipes and causes, not samples

Thing	Stored as
A day of sun	The clock
A swaying tree	Its species recipe, its seed and the wind law
A ferry	A path id and a timetable (32-64 bytes)

Thing	Stored as
A lasting break	Its command (about 32-64 bytes) and the poses of the pieces that last, not their geometry
Lasting inner state (damage, char, a bent rest shape, coats)	Typed records keyed by stable part ids
Passing state (heat that will cool, water that will dry)	Folded by its closed form when the area streams out; never saved
Player poses	Never kept beyond the live window

An empty area folds. When the last player has left and its objects rest, the area keeps a snapshot, its law table and its event queue, and is unloaded. When someone returns at step N:

1. Every law is evaluated at step N, each a closed form.
2. Queued events up to N run in queue order, which is the same on every machine.
3. Their closed-form outcomes are checked by the host before they become lasting.
4. The area unfolds, and its laws run on from there.

Inside an occupied area, results never depend on where players stand: detail changes only by state (sleep) and occupancy (fold and unfold), never by distance.

4.12 Tests

CPU against GPU for equal bits, for every function and law kind; phases against an exact 128-bit rational reference; golden vectors (seeded laws at seeded times, hashed with 64-bit FNV-1a, unchanged in debug and release); a source scan for banned calls and types; each scenario run twice, at several thread counts, saved and reloaded, rolled back and redone, in each build type and platform; GPU kernels at wave sizes 32 and 64; a one-unit change to a constant, which the split detector must find at its first differing tick.

5. Variants

The method may also take any of these forms, alone or together:

Aspect	The method may also
Step	Use another step: 1/240, 1/360, 1/600, 1/960 or 1/1,000 s, any fixed fraction of a second, 1/48,000 s for audio, or a unit that divides common frame rates
Epoch and range	Use another epoch (Unix, GPS, TAI, a project's start, a world's beginning); an unsigned count where no past is needed; a 32-bit count for short-lived systems; TAI or UTC with a leap-second table at the boundary
Fraction	Use 8 to 16 bits or more, nanoseconds, or a 32-bit sub-step fraction everywhere; store the step and the fraction as one 64- or 128-bit tick count
Clocks	Keep any number of clocks (physical, world, match, replay), each a rational map of epoch time, with rates that change at anchors (a speed-up, a pause)
Display time	Read it from any device: a headset, a monitor's next refresh, a projector, an LED wall, a camera shutter (for sub-frame motion-blur samples), an audio buffer, a haptic or motion-platform update
Period and word	Use 2^k steps with 32 – k fraction bits (2^{24} steps with 8 bits, 2^{16} with 16); 64-bit phases (2^{32} steps of 2^{32} ticks, a wrapping 64-bit multiply); 16-bit phases on small devices; a period per law from a set of powers of two; extra fraction bits in n with a wider multiply
Phase forms	Add integer chirps and modulation, such as n × t32 + m × ((t32 × t32) >> s) + φ, wrapping; count phase in other units (2^{-16} turn, binary degrees); compute other exact periods on a GPU with exact integer division, or by a fixed-point reciprocal checked against the CPU
Formats and functions	Use Q1.14 or Q1.62, tables of 1,025 to 65,537 entries, quadratic or cubic interpolation, CORDIC, integer polynomials or quarter tables; make the tables by other exact methods (rational or interval arithmetic, a correctly rounded library run offline), at build or run time, checked by a hash at start; build soft-float or posit twins from the same integer operations; use 64-bit shader integers where offered
Hardware	Run the twins on any CPU (x86-64, ARM, RISC-V, microcontrollers), any GPU (desktop, mobile, integrated, software), DSPs, FPGAs or ASICs, through any interface with 32-bit integers (Vulkan, Direct3D, Metal, OpenGL, OpenCL, WebGPU), or on the CPU alone

Aspect	The method may also
Network and storage	Send laws peer to peer, through a relay, by multicast, with redundancy, or in a replay file; by reference to a library entry, or as a patch to a running law at a given tick; batch, compress and hash them; use two layers or more than three, or gameplay in fixed point; store laws, seeds and causes in databases, files or version control, with closed-form catch-up

Uses outside games. The method may also be used for:

Use	How
Film and animation	Render farms of mixed machines and GPUs get identical procedural water, wind, foliage and crowds at any frame rate, exact motion-blur samples, and re-renders that match years later
Broadcast and live events	Many render engines and LED walls show synchronised procedural graphics from one clock and one set of laws
Distributed simulation and training	Entities move by laws instead of dead-reckoning updates (DIS, HLA)
Digital twins	Periodic machinery and flows kept as laws; unattended areas caught up by closed forms; exact replays
Robotics	Trajectories shared as laws between planners, controllers and simulators; logs in integer time
Financial and scientific replay	Event logs on integer time replayed bit for bit; reproducible GPU kernels with CPU twins
Audio and music	Oscillators whose phase is a product of absolute time, so any sample is computed directly; shared tempo laws
Shows (lighting, fountains, rides, drones, lasers)	Every device evaluates the show's laws on a shared clock
CAD and engineering	Mechanism and kinematic simulations that give the same bits on every engineer's machine
Medicine	Surgical simulators and training replays that match bit for bit; devices in an operating room on one clock
Web, mobile, science	32-bit integer shading in browsers; tides and seasons as exact integer periods on mixed hardware

6. Worked example

10 s and 0.25 ms after the epoch.

1. step = 10 × 480 = 4,800; 0.25 ms is 0.12 step, so fraction = $\lfloor 0.12 \times 2{,}048 \rfloor = 245$.
2. ticks = 4,800 × 2,048 + 245 = 9,830,645, which is also t32.
3. A wave of about 1 Hz: n = round($2^{21} \div 480$) = 4,369 (0.99998 Hz).
4. n × t32 = 42,950,088,005; mod 2^{32} that is 415,045 = phase32 (9.66×10^{-5} turn: the 10 whole cycles are gone).
5. i = 415,045 >> 20 = 0 and f = 415,045. With SIN[0] = 0 and SIN[1] = 1,647,099, the sine is $\lfloor 1{,}647{,}099 \times 415{,}045 \div 2^{20} \rfloor = 651{,}951$, or 6.0718×10^{-4}; the exact sine of $2\pi \times 9.66 \times 10^{-5}$ is 6.0718×10^{-4}.
6. The GPU's split form: fh = 405, fl = 325; $\lfloor (1{,}647{,}099 \times 405 + \lfloor 1{,}647{,}099 \times 325 \div 2^{10} \rfloor) \div 2^{10} \rfloor = \lfloor (667{,}075{,}095 + 522{,}760) \div 1{,}024 \rfloor = 651{,}951$, the same.
7. One step before the epoch (step −1, fraction 0): t32 = $2^{32} - 2{,}048$, which is $-2{,}048 \bmod 2^{32}$ as required, and its 120 Hz tick is −1.

7. Figures



Figure 1. The time value and the words made from it. The step and the fraction are stored; everything else is derived, never summed.

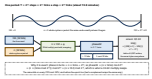


Figure 2. A whole-number phase is one wrapping multiply. A period of 2^{21} steps holds exactly 2^{32} ticks, so n whole cycles a period give the phase $n \times t32 + \varphi$; its top 12 bits pick a sine entry and the next 20 interpolate.

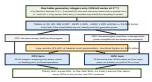


Figure 3. One table generator and two twins. The CPU and the GPU read the same table bytes and law words; the parity test demands equal bits.

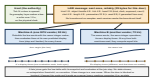


Figure 4. A LAW message and its evaluation on two machines. Sent once; each machine evaluates at its own display times and gets equal bits at equal times.



Figure 5. The three determinism layers, and a break sent as a command. Cosmetic matter is never read by gameplay; debris is each machine's own.

8. References

- Bettner, P., and Terrano, M. (2001). 1500 archers on a 28.8: network programming in Age of Empires and beyond. *Gamasutra*, 22 March 2001.
- Catto, E. (2024). Determinism. box2d.org, August 2024.
- Collange, C., Defour, D., Graillat, S., and Iakymchuk, R. (2015). Numerical reproducibility for the parallel reduction on multi- and many-core architectures. *Parallel Computing*, 49, 83-97.
- Facebook and Oculus (2018). Flicks: a unit of time of 1/705,600,000 s. github.com/OculusVR/Flicks.
- Fiedler, G. (2014). Deterministic lockstep. *Gaffer On Games*.
- GGPO: rollback networking for peer-to-peer games (T. Cannon; released as open source in 2019).
- id Software (1993). *Doom*: 32-bit binary angles and fine sine tables (source code released in 1997).
- IEEE 754-2019. Standard for floating-point arithmetic.
- IEEE 1278.1-2012. Standard for Distributed Interactive Simulation: application protocols.
- IEEE 1588-2019. Standard for a precision clock synchronization protocol.
- Jefferson, D. R. (1985). Virtual time. *ACM Transactions on Programming Languages and Systems*, 7(3), 404-425.
- Khronos Group. The OpenXR specification (predicted display times); the Vulkan specification (precision of SPIR-V instructions); the SPIR-V specification (OpUMulExtended, OpSMulExtended).
- Mills, D., Martin, J., Burbank, J., and Kasch, W. (2010). RFC 5905: Network Time Protocol version 4.
- Monniaux, D. (2008). The pitfalls of verifying floating-point computations. *ACM Transactions on Programming Languages and Systems*, 30(3).
- Nair, V., et al. (2023). Unique identification of 50,000+ virtual reality users from head and hand motion data. *USENIX Security Symposium 2023*.
- Salmon, J. K., Moraes, M. A., Dror, R. O., and Shaw, D. E. (2011). Parallel random numbers: as easy as 1, 2, 3. *SC11*.
- Stallone, M. (2018). 8 frames in 16 ms: rollback networking in Mortal Kombat and Injustice 2. *Game Developers Conference 2018*.
- Stam, J. (1997). Stochastic dynamics: simulating the effects of turbulence on flexible structures.

Computer Graphics Forum, 16(3), 159-164.
- Steele, G. L., Lea, D., and Flood, C. H. (2014). Fast splittable pseudorandom number generators. *OOPSLA 2014*.
- Tessendorf, J. (2002). Simulating ocean water. *SIGGRAPH course notes*.
- Tierney, J., Rader, C. M., and Gold, B. (1971). A digital frequency synthesizer. *IEEE Transactions on Audio and Electroacoustics*, AU-19(1), 48-57.

© 2026 Kyle Kimberlin. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/